

JAVA

Expressões, Statements & Métodos em Java

Do conceito de método até a sobrecarga e as boas práticas de design

Apostila completa sobre métodos em Java, com exemplos práticos de parâmetros, retorno e sobrecarga.

CONTEÚDO DA APOSTILA

- 01 Unidades de Código Java
- 02 Introdução aos Métodos
- 03 Parâmetros vs. Argumentos
- 04 Tipo de Retorno e void
- 05 if-then-else Completo
- 06 Sobrecarga de Métodos

Sumário

0
1

Unidades de Código Java

Expressões, statements, blocos e a hierarquia do código

0
2

Introdução aos Métodos

O que são, por que existem e sua estrutura básica

0
3

Parâmetros vs. Argumentos

A diferença de terminologia e as regras de chamada

0
4

Tipo de Retorno e void

Como declarar o retorno e a assinatura de um método

0
5

if-then-else Completo

if simples, if-else e o encadeamento com else-if

0
6

Sobrecarga de Métodos

Overloading, assinatura e as regras de validade

01

Unidades de Código Java

Expressões, statements e blocos — a hierarquia que organiza todo código Java

Do menor pedaço ao maior

Escrever Java se parece com escrever um documento bem estruturado: existem unidades menores que se combinam para formar unidades maiores. No nível mais básico está a expressão, um trecho de código que computa e devolve um valor. Expressões se combinam para formar statements — instruções completas que terminam em ponto e vírgula. Statements se agrupam em blocos delimitados por chaves, blocos se organizam dentro de métodos, e métodos vivem dentro de classes.

```
// EXPRESSÃO – computa e devolve um valor:
3 + 2 // devolve 5
quantidadeItens > 0 // devolve true ou false
prato.length() // devolve um int (o tamanho do texto)

// STATEMENT – instrução completa, termina com ;
int numeroMesa = 5; // declaration statement
numeroMesa = numeroMesa + 1; // expression statement
System.out.println("Pedido enviado"); // method call statement

// BLOCO – conjunto de statements entre chaves:
if (numeroMesa > 0) {
    System.out.println("Mesa válida");
    numeroMesa = 0;
}
```

DICA

O compilador Java ignora completamente espaços, tabulações e linhas em branco. Use esse espaço livre a seu favor para deixar o código mais legível — a convenção mais seguida pela comunidade é o Google Java Style Guide.

02

Introdução aos Métodos

Agrupando código executável para eliminar repetição

O problema que os métodos resolvem

Um método reúne um trecho de código executável sob um único nome, para que ele possa ser reaproveitado sempre que necessário. Sem métodos, o mesmo trecho de lógica precisaria ser copiado e colado em cada lugar onde fosse usado — e qualquer correção futura teria que ser replicada em todas essas cópias, multiplicando o risco de erro. Esse princípio de escrever uma vez e usar muitas vezes é conhecido pela sigla DRY (Don't Repeat Yourself).

Um mesmo método pode ser chamado repetidamente com argumentos diferentes, produzindo resultados diferentes a cada chamada — é essa flexibilidade que faz do método a unidade básica de reuso em Java.

```
// Método mais simples possível – sem parâmetros, sem retorno:
public static void prepararPedido() {
    System.out.println("Preparando o pedido...");
}

// Chamando o método:
prepararPedido();

// Método com parâmetros e retorno:
public static double calcularConta(boolean temSobremesa, int qtdPratos, double
gorjeta) {
    if (temSobremesa) {
        return qtdPratos * 35.0 + gorjeta;
    }
    return qtdPratos * 35.0;
}

// Chamando com argumentos reais:
double total = calcularConta(true, 3, 15.0); // total = 120.0
```

03

Parâmetros vs. Argumentos

Uma distinção de terminologia que importa para a comunicação técnica

Duas palavras, dois momentos diferentes

Parâmetro é a variável que aparece na declaração do método — descreve o que o método espera receber. Argumento é o valor concreto passado no momento em que o método é chamado. Embora no dia a dia os dois termos às vezes sejam usados como sinônimos, saber diferenciá-los ajuda bastante ao ler documentação técnica ou discutir código com outros desenvolvedores.

```
// PARÂMETROS aparecem na DECLARAÇÃO:
public static void fazerPedido(String prato, int quantidade) {
    System.out.println(quantidade + "x " + prato + " enviado para a cozinha.");
}

// ARGUMENTOS aparecem na CHAMADA:
fazerPedido("Risoto de Camarão", 2); // "Risoto..." e 2 são os argumentos

// Erros comuns ao chamar um método:
// fazerPedido(2, "Risoto de Camarão"); // ordem trocada!
// fazerPedido("Risoto de Camarão"); // faltam argumentos!
// fazerPedido("Risoto de Camarão", 2, 5); // argumentos além do esperado!
```

ATENÇÃO

Diferente de Python ou JavaScript, o Java não suporta parâmetros com valor padrão. Todo argumento declarado precisa ser passado em toda chamada. Quando valores padrão fazem falta, a solução em Java é criar sobrecargas do método.

04

Tipo de Retorno e void

Declarando o que um método devolve — e quando ele não devolve nada

void e a obrigatoriedade do return

O tipo de retorno é declarado logo antes do nome do método. Quando ele é **void**, o método não devolve dado nenhum — apenas executa uma ação. Para qualquer outro tipo (int, double, String, boolean...), o método é obrigado a ter um **return** com um valor compatível em absolutamente todos os caminhos possíveis de execução — inclusive dentro de blocos if e else.

```
// void – nenhum valor é devolvido:
public static void imprimirComanda(String item) {
    System.out.println(item);
    // return; // opcional em void – pode encerrar antes do fim
}

// String – retorno presente em todos os caminhos:
public static String classificarPedido(int quantidadeItens) {
    if (quantidadeItens >= 5) return "Pedido grande";
    else return "Pedido pequeno"; // TODOS os caminhos retornam algo
}

// Padrão recomendado – variável de resultado com um único return:
public static double maiorConta(double contaA, double contaB) {
    double resultado = contaA; // valor padrão
    if (contaB > contaA) resultado = contaB;
    return resultado; // único return, no final
}
```

DICA

Uma boa prática consolidada é declarar um valor padrão logo no início do método e usar um único return no final, em vez de vários returns espalhados. O fluxo fica mais previsível, mais fácil de debugar e mais fácil de testar.

05

if-then-else Completo

As três formas de estruturar uma decisão, incluindo múltiplas alternativas

if simples, if-else e o encadeamento com else-if

O Java oferece três variações da estrutura condicional. O if simples executa ou não executa um bloco, sem alternativa. O if-else escolhe entre duas alternativas — uma coisa ou outra. E o if-else-if encadeado permite avaliar múltiplas condições em sequência, sendo o else final (sem condição própria) opcional e responsável por capturar qualquer caso que não se encaixe nas condições anteriores.

```
// 1. if simples:
if (quantidadeItens >= 10) { System.out.println("Pedido grande"); }

// 2. if-else:
if (quantidadeItens >= 5) { System.out.println("Pedido médio"); }
else { System.out.println("Pedido pequeno"); }

// 3. if-else if-else (múltiplas condições em cadeia):
if (quantidadeItens >= 10) { System.out.println("Pedido grande"); }
else if (quantidadeItens >= 5) { System.out.println("Pedido médio"); }
else if (quantidadeItens >= 1) { System.out.println("Pedido pequeno"); }
else { System.out.println("Nenhum item no pedido"); }
```

06

Sobrecarga de Métodos (Overloading)

Mesmo nome, parâmetros diferentes — o polimorfismo resolvido em tempo de compilação

Como o compilador escolhe qual versão chamar

Sobrecarga é a possibilidade de ter mais de um método com o mesmo nome dentro da mesma classe, desde que os parâmetros sejam diferentes — seja em tipo, em quantidade ou na ordem. É o compilador quem decide, analisando os argumentos passados em cada chamada, qual versão do método deve ser executada. Esse mecanismo é chamado de polimorfismo de compilação, porque a decisão acontece antes mesmo do programa rodar.

```
// Versão 1: recebe apenas o subtotal
public static double calcularTotal(double subtotal) {
    return subtotal * 1.10; // taxa de serviço de 10%
}

// Versão 2: recebe o subtotal e uma gorjeta extra
public static double calcularTotal(double subtotal, double gorjetaExtra) {
    double totalComTaxa = calcularTotal(subtotal); // chama a versão 1!
    return totalComTaxa + gorjetaExtra;
}

// O Java identifica qual usar pelos argumentos da chamada:
double contaA = calcularTotal(100.0); // 110.0
double contaB = calcularTotal(100.0, 20.0); // 130.0
```

ATENÇÃO — O que NÃO conta como assinatura

O tipo de retorno e os nomes dos parâmetros não fazem parte da assinatura de um método. Criar duas sobrecargas que diferem apenas no tipo de retorno, ou apenas no nome dos parâmetros, resulta em erro de compilação — a diferença precisa estar no tipo, na quantidade ou na ordem dos parâmetros.

Boas práticas no design de métodos

Alguns princípios ajudam a manter métodos saudáveis ao longo do tempo: cada método deve fazer uma única coisa, e fazê-la bem — o princípio da responsabilidade única. Métodos curtos, idealmente com menos de 20 ou 30 linhas, são mais fáceis de entender e de testar. Nomes descritivos como `calcularMedia()` comunicam muito mais do que abreviações como `cm()` ou `x()`. Entradas de métodos públicos merecem validação — nunca é seguro confiar cegamente nos argumentos recebidos. E, como já visto, um único `return`

no final costuma facilitar o debug.

Fim de apostila

Métodos são a base de todo código Java bem organizado.

Dominar parâmetros, tipos de retorno e sobrecarga é o que permite construir programas modulares, reutilizáveis e fáceis de manter.

JavaStudiesHub